space of states to the space of actions. In other words, you give it a state vector and it returns an action to take. In chapter 4 we used a neural network as a policy function and directly trained it to output the best actions. In Q-learning, we have the intermediate step of first computing action values (Q values) for a given state, and then we use those action values to decide which action to take. So in Q-learning, the policy function takes in Q values and returns an action.

---

definition

The softmax function is defined mathematically as

$$P_t(a) = \frac{\exp(q_t(a)/\tau)}{\sum_{i=1}^{n} \exp(q_t(i)/\tau)},$$

where $P_t(a)$ is the probability distribution over actions, $q_t(a)$ is a Q-value vector, and τ is the temperature parameter.

As a reminder, the softmax function takes in a vector with arbitrary numbers and then "normalizes" this vector to be a probability distribution, so that all the elements are positive and sum to 1, and each element after the transformation is proportional to the element before the transformation (i.e., if an element was the largest in the vector, it will be assigned the largest probability). The softmax function has one additional input, the temperature parameter, denoted with the Greek symbol tau, τ.

If the temperature parameter is large, it will minimize the difference in probabilities between the elements, and if the temperature is small, differences in the input will be magnified. For example, the vector `softmax([10,5,90], temp=100) = [0.2394, 0.2277, 0.5328]` and `softmax([10,5,90], temp=0.1) = [0.0616, 0.0521, 0.8863]`. With a high temperature, even though the last element, 90, is 9 times larger than the second-largest element, 10, the resulting probability distribution assigns it a probability of 0.53, which is